Daily Organizer — Complete Documentation

> Written for a frontend developer (HTML/CSS/JS + a little Angular) who wants to understand how the full project works — frontend, backend, database, and everything in between. No prior backend/API experience assumed.

Table of Contents

13. Glossary

---

1. What is this project?

Purpose

Daily Organizer is a single-user personal productivity app that consolidates the things you'd otherwise spread across five separate tools — a task manager, habit tracker, journal, calendar, and a freelance CRM — into one private workspace.

It is **deliberately not a team / shared product**. Every record is scoped to one account: your tasks, your habits, your journal, your client projects, your payment history. There's no sharing, no workspace concept, no admin roles. This makes the data model simple (everything has a `userId` foreign key) and the privacy story easy to reason about.

It is **hostable on free tiers** — Firebase Realtime Database (free up to 1 GB), Render (free backend instance), Vercel (free frontend hosting). The app is built so the same person who owns the data also runs the infrastructure.

Who this is for

The original author is a freelance developer in India who wanted to:

- Plan tasks and events alongside a calendar (without leaking personal events into a team workspace)
- Track recurring habits with streaks
- Reflect on each day in a private journal
- Manage a freelance project pipeline — quotes, deadlines, payments
- See a snapshot of "today" without opening five apps

If your needs overlap, this is for you. If you need team collaboration, a different product (Notion, Asana, etc.) is a better fit.

What it does — full feature list

Module	What it does
Dashboard	Greeting + today's snapshot: stats, schedule, goal progress, and the Daily Routine widget (progress ring + 30-day consistency heatmap + interactive habit checklist)
My Plans (Tasks)	All scheduled items — tasks, trips, journeys, meals, meetings, events, reminders — with type chips, status / priority filters, 15-per-page pagination, and DONE rows auto-sorted to the bottom
Goals	Long-term goals → milestones → mini-goals, with auto-calculated progress %, reorderable milestones, resources list, target date
Daily Routine	Per-weekday recurring habits with time intervals, current streak, 30-day heatmap, date navigation to backfill missed days, optimistic toggle for instant UI

Module	What it does
(Habits)	
Journal	One reflection entry per day, with mood emoji, title, body, streak of consecutive days journaled, and browse-past-entries list
Projects (freelance)	Project pipeline (LEAD → QUOTED → IN_PROGRESS → DELIVERED → PAID / LOST / ON_HOLD), per-project payments, deadlines, progress %, portfolio links, outstanding-balance and monthly-income stats
Trips	Kanban board for travel plans (Bucket List → Planning → Booked → Visited). Drag-and-drop between columns. Columns fill viewport height with internal card scrolling. Cards sorted by: Bucket List → A-Z, Planning/Booked → nearest date first, Visited → newest first. Year pill on each dated card. Trips with dates auto-sync to calendar and trigger habit trip-day exclusion. Past BOOKED trips auto-promote to Visited
Buy List	Kanban board for things to remember to buy (Want → Considering → Bought → Skipped). Drag-and-drop between columns. Viewport-height columns with internal card scrolling. Auto-stamps "bought on" date when moved to Bought
Calendar	Monthly grid showing plans + Google Calendar events (festivals, holidays, restaurant/movie bookings) plotted by date; weekends/birthdays/holidays/leaves/✈trip overlays. Custom month-picker dropdown (click the month title) lets you jump to any month/year without stepping through one at a time
Categories	User-defined color-coded labels for grouping tasks
Google Calendar sync	Two-way: pushes app plans to your primary Google calendar, AND pulls events from all your subscribed calendars (holidays, birthdays, third-party bookings) into the app's calendar view
Profile	Edit display name, email, password, employment + office hours, weekend days, date of birth
Dark / Light theme	Auto-detect system preference, manual toggle, persisted to localStorage
Auth	Email/password register + login, JWT access + refresh token rotation, auto-redirect on expired session

How it's organized

It's a **full-stack** app: there's a **frontend** (what you see in the browser), a **backend** (a separate program running on a server), and a **database** (where data is stored permanently). All three are in this repo:

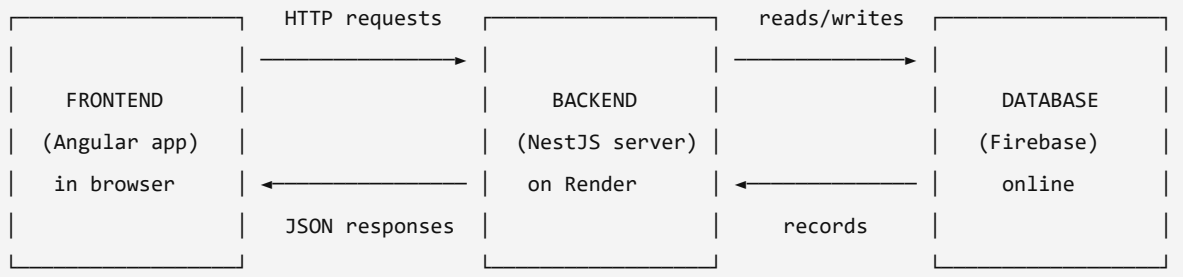
```
Daily Organiser/
├── frontend/     ← Angular app (what runs in your browser)
├── backend/      ← NestJS API server (runs on Node.js)
└── README.md     ← project entry doc
```

---

2. The big picture: how a modern web app works

If you've only built static HTML/CSS/JS sites, the jump to "full-stack" can feel mysterious. Here's the mental model.

Three independent programs that talk to each other



Each box is a **separate program**:

- **Frontend** = the HTML/CSS/JS files Angular generates. Lives in the user's browser. Has no direct access to the database. Can only ask the backend for data via HTTP.
- **Backend** = a Node.js program that runs on a server somewhere (locally for dev, Render in production). It listens for HTTP requests, validates them, reads/writes the database, and sends back JSON responses.
- **Database** = a remote system that just stores data. Frontend never talks to it directly — only the backend has the credentials.

The frontend and backend talk over **HTTP** (the same protocol your browser uses to load any web page). The backend talks to Firebase using Firebase's own SDK.

Why three layers instead of one?

Security. If the frontend talked to the database directly, the database password would be in the browser — anyone could steal it. The backend keeps the secrets and acts as a gatekeeper.

Flexibility. You could swap the frontend for a mobile app, or swap the database for PostgreSQL, without rewriting the other layers. The HTTP contract stays the same.

Logic. Things like "only this user can edit this task" need to be enforced somewhere trusted. The browser can't be trusted (users can modify it), so the backend is where rules live.

---

3. The three layers explained

HTTP: the language between frontend & backend

HTTP is a simple request/response protocol. Every interaction is:

1. The frontend sends a **request** (a verb + URL + optional body).
2. The backend sends back a **response** (a status code + body).

Common verbs:

Verb	Meaning	Example
`GET`	"give me this data"	`GET /api/tasks` → list all tasks
`POST`	"create something new"	`POST /api/tasks` with body `{title: "Buy milk"}`
`PATCH`	"update part of something"	`PATCH /api/tasks/abc123` with body `{status: "DONE"}`
`DELETE`	"remove something"	`DELETE /api/tasks/abc123`

Common status codes:

- `200` OK — request succeeded
- `201` Created — request succeeded and made something new
- `400` Bad Request — your request body was wrong
- `401` Unauthorized — you're not logged in or your token is invalid
- `403` Forbidden — you're logged in but not allowed to do this
- `404` Not Found — the thing you asked for doesn't exist
- `500` Server Error — backend bug

The body of requests and responses is **JSON**:

```
{ "id": "abc", "title": "Buy milk", "status": "DONE" }
```

JSON is just a text format that looks like a JavaScript object. Both frontend and backend can parse/serialize it.

API: a contract between frontend & backend

"API" is just the **list of HTTP endpoints the backend exposes** and what each one does. For example:

- `GET /api/habits` returns an array of habits
- `POST /api/habits` accepts a body like `{title, weekdays, ...}` and creates one

In this project, the API base URL is `http://localhost:3000/api` in dev (see `backend/src/main.ts`).

Database: where data actually lives

This project uses **Firebase Realtime Database** — a hosted, JSON-tree database from Google. It's not SQL; it's just a big tree of JSON. The shape looks roughly like:

```
firebase-root/
├── users/
│   └── <userId>/ { email, username, passwordHash, ... }
├── tasks/
│   └── <taskId>/ { title, userId, status, ... }
├── goals/
│   └── <goalId>/ { title, userId, progress, ... }
├── milestones/
│   └── <milestoneId>/ { goalId, status, ... }
├── habits/
│   └── <habitId>/ { title, weekdays, userId, ... }
├── habitCheckins/
│   └── <checkinId>/ { habitId, userId, date, ... }
└── categories/
    └── <categoryId>/ { name, color, userId }
```

Every record has its own unique ID. Records reference each other by ID (e.g., a task has a `userId` field pointing to which user owns it).

The backend uses Firebase's Admin SDK to read/write this tree. See `backend/src/prisma/firebase.service.ts` — that's the wrapper that exposes `get`, `set`, `update`, `remove`, `getList` methods used by every backend service.

> **Why is the file called `prisma`?** Historical. The project originally used Prisma + PostgreSQL. When it migrated to Firebase, the folder and module names stayed for compatibility. The actual implementation is purely Firebase.

---

4. Tech stack & why each piece exists

Frontend: Angular 19

Angular is a JavaScript framework that helps build big single-page apps (SPAs). Why not just HTML/CSS/JS?

Problem	Angular's answer
Sharing logic across many pages without copy-paste	**Components** — reusable building blocks (`<app-sidebar>`, `<app-modal>`)
Updating the DOM when data changes	**Data binding** — `{{ variable }}` in the template updates automatically when `variable` changes
Talking to a backend	**HttpClient service** — handles `fetch` / `axios` -style calls cleanly
Routing without page reloads	**Router** — `/dashboard` vs `/tasks` switches without refresh
Shared state between unrelated components	**Services + RxJS** — a singleton class others inject

Problem	Angular's answer
Validating forms	**Reactive Forms** — declarative form objects with validators

Specifically this project uses **standalone components** (the newer Angular 19 style — no NgModule boilerplate) and **signals** (`signal(...)` , `computed(...)`) for reactive state.

Backend: NestJS 11

NestJS is a Node.js framework for building backend APIs. It's heavily inspired by Angular — same idea of decorators, dependency injection, modules.

Problem	NestJS answer
Defining HTTP endpoints	**Controllers** with `@Get()` , `@Post()` decorators on methods
Reusing business logic across controllers	**Services** — classes with `@Injectable()` , automatically wired up
Validating incoming request bodies	**DTOs + class-validator** decorators (`@IsString()` , `@IsOptional()`)
Organizing related code	**Modules** — `TasksModule` bundles `TasksController + TasksService`
Auth & permissions	**Guards** — run before a controller method, can deny access

A typical request flow inside NestJS:

1. HTTP request arrives at a `@Controller('xyz')` -decorated class
2. Global `JwtAuthGuard` checks for a valid JWT in the `Authorization` header
3. `ValidationPipe` validates the body against a DTO
4. The matching method runs (e.g., `findAll(userId)`)
5. It calls a `@Injectable()` service which talks to Firebase
6. Returned value is auto-serialized to JSON and sent back

Database: Firebase Realtime Database

A hosted NoSQL JSON-tree database. Pros:

- Zero setup (no installing/running a DB server)
- Free tier suitable for a personal app
- Simple read/write API

Cons:

- No SQL joins — you fetch lists and filter in code
- Lacks proper indexes — fine for small datasets, slow at scale
- Not relational — easy to end up with inconsistent data if you're not careful

For this app's scale (one user, a few hundred records), the tradeoff is fine.

Authentication: JWT (JSON Web Tokens)

When a user logs in, the backend gives them a **token** (a long string). The frontend stores it and sends it with every future request. The backend verifies the token to know which user is calling.

A JWT looks like:

```
eyJhbGciOiJIUzI1NiIs...header.eyJzdWIiOiJ1c2VyMTIz....payload.signature
```

It's three base64-encoded parts: header, payload (contains user id + expiry), and a signature. The backend can verify the signature with a secret key (`JWT_SECRET` in `.env`) and trust the payload without a database lookup.

This app uses **two** tokens:

- **Access token** — short-lived (15 min), sent on every request
- **Refresh token** — long-lived (7 days), used only to get a new access token

When the access token expires, the frontend silently calls `/api/auth/refresh` with the refresh token to get a new access token. If that fails too (refresh token also expired), the user is logged out.

---

5. Project structure (annotated)

Frontend (`frontend/src/app/`)

```
app/
├── app.config.ts               ← bootstrap config: router + http + interceptors
├── app.routes.ts               ← all URL → component mappings (lazy-loaded)
├── app.component.ts            ← root component (just <router-outlet>)
│
├── core/                       ← cross-cutting concerns (loaded once, app-wide)
│   ├── guards/
│   │   ├── auth.guard.ts         ← blocks unauth users from /dashboard etc.
│   │   └── no-auth.guard.ts      ← blocks logged-in users from /auth/login
│   ├── interceptors/
│   │   └── auth.interceptor.ts   ← attaches "Bearer <token>" to outgoing requests,
│   │                               silently refreshes on 401, redirects to login on failure
│   ├── models/                 ← TypeScript interfaces (just type definitions)
│   │   ├── user.model.ts
│   │   ├── task.model.ts
│   │   ├── goal.model.ts
│   │   ├── habit.model.ts
│   │   ├── journal.model.ts
│   │   ├── project.model.ts
│   │   ├── trip.model.ts
│   │   ├── buy-item.model.ts
```

```
│   │   └── category.model.ts
│   └── services/                ← singleton classes injectable everywhere
│       ├── auth.service.ts        ← login/register/refresh, current user signal
│       ├── token-storage.service.ts ← localStorage wrapper for the tokens
│       ├── theme.service.ts       ← dark/light mode toggle
│       ├── toast.service.ts       ← in-app notification system
│       └── google-calendar.service.ts
│
├── shared/                      ← reusable UI components used across features
│   └── components/
│       ├── layout/              ← the app shell: sidebar + topbar + <router-outlet>
│       ├── sidebar/             ← left nav, theme toggle, user block
│       ├── topbar/              ← optional top bar (not currently rendered)
│       ├── modal/               ← dialog wrapper used by all forms
│       ├── confirm-dialog/      ← "Are you sure?" reusable popup
│       ├── toast-container/     ← renders toasts from ToastService
│       └── category-manager/    ← category CRUD popup
│
└── features/                    ← one folder per page/feature
    ├── auth/login | register/
    ├── dashboard/               ← "Dashboard" page
    ├── tasks/
    │   ├── task.service.ts      ← talks to /api/tasks
    │   ├── task-list/           ← "My Plans" page
    │   ├── task-form/           ← form rendered inside the modal
    │   └── task-detail/         ← /tasks/:id detail page with timer
    ├── goals/
    │   ├── goal.service.ts
    │   ├── goal-list/
    │   ├── goal-form/
    │   └── goal-detail/         ← milestones + mini-goals UI
    ├── habits/
    │   ├── habit.service.ts
    │   ├── habit-list/          ← "Daily Routine" page
    │   └── habit-form/
    ├── journal/
    │   ├── journal.service.ts
    │   └── journal-page/        ← "Journal" page (today's reflection + past entries)
    ├── projects/
    │   ├── project.service.ts
    │   ├── project-list/        ← pipeline view with stats and status filter chips
    │   ├── project-form/        ← create/edit project modal
    │   └── project-detail/      ← project info, payments, edit, status changer
    ├── trips/
    │   ├── trip.service.ts
    │   ├── trip-board/          ← Kanban board with drag-and-drop
    │   └── trip-form/           ← create/edit/delete trip modal
    ├── buy-list/
```

```
    │   ├── buy-list.service.ts
    │   ├── buy-list-board/      ← Kanban board with drag-and-drop
    │   └── buy-list-form/       ← create/edit/delete item modal
    ├── calendar/                ← monthly grid (renders app plans + Google events)
    ├── categories/              ← category service only (UI is in shared/)
    └── profile/                 ← edit profile + change password
```

Each feature folder is **self-contained**: component + service + (optional) form. The service inside each feature is what talks to the backend.

Backend (`backend/src/`)

```
src/
├── main.ts                      ← server entry: creates app, enables CORS, listens on port 3000
├── app.module.ts                ← root module, imports every feature module
│
├── auth/                        ← register/login/refresh/logout
│   ├── auth.module.ts             ← bundles controller + service + JWT strategies
│   ├── auth.controller.ts         ← HTTP endpoints (@Post('register'), etc.)
│   ├── auth.service.ts            ← business logic: hash password, generate tokens
│   ├── strategies/
│   │   ├── jwt.strategy.ts        ← validates access tokens on every request
│   │   └── jwt-refresh.strategy.ts ← validates refresh tokens on /auth/refresh
│   ├── guards/                  ← JwtAuthGuard (global), JwtRefreshGuard
│   └── dto/                     ← request body shapes (RegisterDto, LoginDto)
│
├── users/                       ← /api/users/profile GET/PATCH
├── tasks/                       ← /api/tasks/* CRUD + timer endpoints
├── goals/                       ← /api/goals/* + nested milestones + mini-goals
├── habits/                      ← /api/habits/* + check-ins, streaks, heatmap
├── journal/                     ← /api/journal/* — daily reflections (one entry per date)
├── projects/                    ← /api/projects/* + nested payments (freelance tracker)
├── trips/                       ← /api/trips/* — Kanban for travel plans + auto-syncs to calendar
├── buy-list/                    ← /api/buy-list/* — Kanban for things to buy
├── categories/                  ← /api/categories/* CRUD
├── dashboard/                   ← /api/dashboard/stats|activity|calendar (aggregations)
├── google-calendar/             ← /api/google/* OAuth + bidirectional sync
│
├── common/                      ← shared utilities
│   └── decorators/
│       ├── public.decorator.ts    ← @Public() — skip JWT guard for register/login/refresh
│       └── current-user.decorator.ts ← @CurrentUser('id') — extract user from JWT
│
└── prisma/                      ← name is historical; this is the Firebase wrapper
    ├── prisma.module.ts           ← exposes FirebaseService globally
    └── firebase.service.ts        ← .get(), .set(), .update(), .remove(), .getList()
```

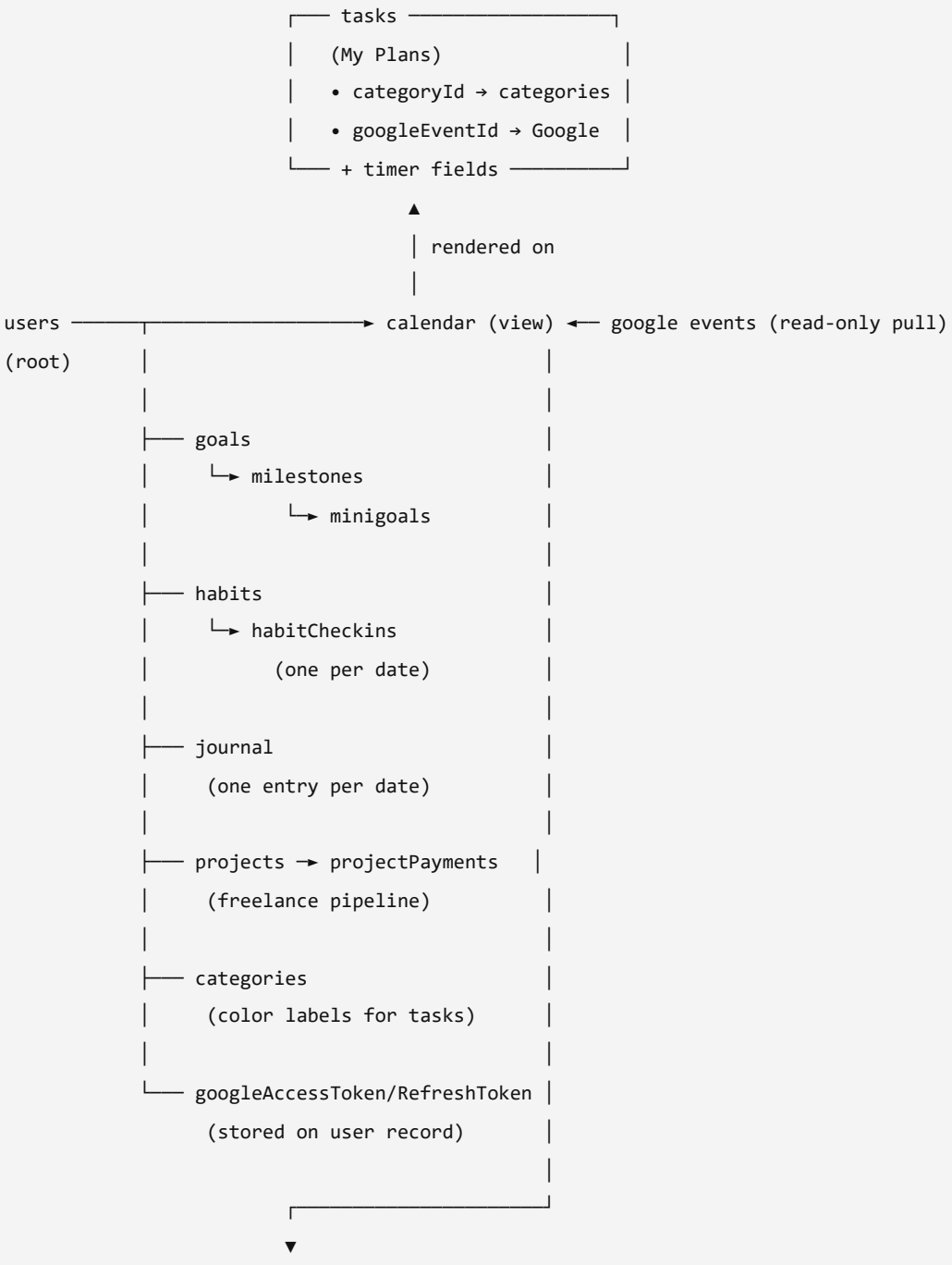
Every feature module follows the same skeleton:

- A `XxxModule` (just wires things up)
- A `XxxController` (HTTP routing — small file)
- A `XxxService` (business logic + Firebase reads/writes — the meat)
- `dto/` folder with classes describing what request bodies should look like

How the modules connect to each other

Every record in this app is owned by exactly one user. Modules don't share data across users — they share **logical relationships** *within* one user's account, expressed as foreign-key IDs.

Entity relationships (Firebase collections)



```
                        dashboard (view) ——— aggregates from:
                                              tasks, goals, habits,
                                              and reads habit$ stream
```

Foreign-key conventions

Every child entity stores a reference to its parent as a string ID field. We use Firebase like a small relational DB without joins — we filter in JavaScript instead of using SQL.

Child	Parent reference	Joined where
`tasks.categoryId`	`categories/<id>`	`TasksService` attaches the full category on read
`tasks.googleEventId`	Google event ID	Used to dedupe Google reads + propagate updates/deletes
`milestones.goalId`	`goals/<id>`	`GoalsService.findOne` joins via filter
`minigoals.milestoneId`	`milestones/<id>`	Joined per-milestone inside `attachMilestones`
`habitCheckins.habitId`	`habits/<id>`	`enrich(habit)` filters check-ins for streak + heatmap
`journal.userId` + `journal.date`	(composite uniqueness)	Service ensures one entry per (user, date)
`projectPayments.projectId`	`projects/<id>`	`enrich(project)` filters payments + computes balance
`trips.taskId`	`tasks/<id>`	When a trip moves to Planning/Booked/Visited with a startDate, the trips service creates a matching `type='trip'` task and stores its ID. Calendar + habit trip-exclusion pick up that task automatically
`buyItems` (no FKs)	—	Buy items are standalone — no relationships

What "joining" looks like in code

Because Firebase has no joins, every multi-collection read fetches each collection separately and combines them in memory. Example from `HabitsService.findAll`:

```
const [habits, checkins] = await Promise.all([
  this.firebase.getList('habits'),
  this.firebase.getList('habitCheckins'),
]);
const userHabits = habits.filter(h => h.userId === userId && !h.archived);
const userCheckins = checkins.filter(c => c.userId === userId);
return userHabits.map(h => this.enrich(h, userCheckins, todayKey));
```

The `enrich()` function then walks the check-ins for each habit to compute streak + 30-day history.

For a few hundred records per user this is fast (~10–50 ms). At scale you'd index in Firebase (`.indexOn` rules) or move to a relational DB.

Views that aggregate across modules

Two pages don't own their own data — they reach into multiple modules:

- **Dashboard** (`/dashboard`):
  - Calls `GET /api/dashboard/stats` (aggregates tasks + goals)
  - Calls `TaskService.getToday()` for today's plans timeline
  - Calls `GoalService.loadAll()` to find active goals
  - Subscribes to `HabitService.habits$` for the Daily Routine widget (so a toggle on the habits page instantly reflects on the dashboard)
- **Calendar** (`/calendar`):
  - Calls `GET /api/dashboard/calendar?year=&month=` for the month's plans
  - Calls `GET /api/google/events?from=&to=` for external Google events when connected
  - Reads `userProfile` for weekend / birthday / office-hours overlays
  - Reads localStorage for holiday / leave overrides

The shared `BehaviorSubject`s inside service classes (e.g., `habits$` , `tasks$` , `projects$` , `entries$`) are the cross-module sync mechanism. Any component that subscribes sees updates from any other component that mutates via the same service.

What is *not* connected

- Tasks and habits are independent: completing a habit does not check off any task, and vice versa
- Goals and habits are independent: a habit isn't a milestone of a goal (deliberate — keeps the schemas separate)
- Projects and tasks are independent: a project's deliverables aren't represented as tasks (could be a future feature)
- Journal entries aren't auto-linked to that day's tasks (also a possible future feature)

This is intentional — we keep modules orthogonal so each can evolve without rippling into the others.

---

6. Following a click: full data flow example

Let's trace **"User checks off the Wakeup habit on the dashboard"** through every layer.

Step 1 — User clicks the checkbox

In the browser, the user sees this in dashboard.component.ts:

```
<button class="th-row" (click)="toggleHabit(h)"> ... </button>
```

The `(click)` binding tells Angular: when this button is clicked, call `toggleHabit(h)` on the component class.

Step 2 — Component delegates to the service

`toggleHabit()` in the dashboard component:

```
toggleHabit(h: Habit) {
  this.habitService.toggleToday(h.id).subscribe({ ... });
}
```

It hands off to `HabitService` — a singleton injected via `inject(HabitService)`. The service is shared by both the dashboard and the habits page; that's how the BehaviorSubject keeps them in sync.

Step 3 — Service does an optimistic update, then HTTP

habit.service.ts:

```
toggleToday(id: string) {
  const original = this.habits$.value.find((h) => h.id === id);
  if (original) {
    // Flip UI state instantly, before waiting for the server.
    this.replaceHabit(this.optimisticToggleToday(original));
  }
  return this.http.post<Habit>(
    `${this.base}/${id}/checkin?today=${this.localTodayKey()}`,
    {}
  ).pipe(
    tap((u) => this.replaceHabit(u)),          // replace with server's authoritative version
    catchError((err) => {
      if (original) this.replaceHabit(original); // revert on failure
      return throwError(() => err);
    }),
  );
}
```

Key things happening here:

- **Optimistic update**: the local in-memory `habits$` BehaviorSubject is flipped immediately so the UI updates the moment you click — no waiting for the server.
- **HTTP POST**: a request is sent to `http://localhost:3000/api/habits/<id>/checkin?today=2026-05-23`.
- **Replace**: when the server responds, the local state is replaced with the real one (in case the server computed something the optimistic update couldn't, like the streak).
- **Revert on error**: if the network fails, the UI flips back.

Step 4 — HTTP interceptor adds the auth token

Before the request actually leaves the browser, auth.interceptor.ts sees it:

```
const token = tokens.getAccessToken();
const authReq = token
  ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
  : req;
return next(authReq).pipe(...);
```

So the request that actually goes out has a header:

```
Authorization: Bearer eyJhbGc...
```

The interceptor will also catch 401 responses and try to refresh the token automatically — see Authentication explained.

Step 5 — Browser sends the request, NestJS receives it

The request lands on the NestJS server. Order of processing:

1. **CORS middleware** — checks the request is allowed from `http://localhost:4200`. See `app.enableCors(...)` in `main.ts`.
2. **Global JWT guard** (`JwtAuthGuard` registered in `app.module.ts`) — verifies the `Authorization` header. It uses Passport JWT strategy to decode the token and load the user. If invalid, returns 401.
3. **Routing** — NestJS finds the controller method matching `POST /api/habits/:id/checkin`. That's habits.controller.ts:

   ```
   @Post(':id/checkin')
   checkinToday(
     @CurrentUser('id') userId: string,
     @Param('id') id: string,
     @Query('today') today?: string,
   ) {
     return this.habitsService.toggleCheckin(userId, id, today, today);
   }
   ```

4. **Decorators inject params**: `@CurrentUser('id')` extracts the user id from the JWT-validated request; `@Param('id')` reads the URL segment; `@Query('today')` reads the query string.
5. **Delegate to service**: hands off to `HabitsService.toggleCheckin(...)`.

Step 6 — Service reads/writes Firebase

habits.service.ts:

```
async toggleCheckin(userId, habitId, date, today) {
  await this.ensureOwnership(userId, habitId);          // 403 if user doesn't own this habit
  const todayKey = this.resolveToday(today);
  const targetDate = date ?? todayKey;

  const checkins = await this.firebase.getList<...>('habitCheckins');
  const existing = checkins.find((c) =>
    c.habitId === habitId && c.date === targetDate && c.userId === userId
  );

  if (existing) {
    await this.firebase.remove(`habitCheckins/${existing.id}`); // un-check
  } else {
    const id = randomUUID();
    await this.firebase.ref(`habitCheckins/${id}`).set({
      id, habitId, userId, date: targetDate, completedAt: new Date().toISOString(),
    });
  }

  // Re-fetch habit + checkins, compute streak/scheduled/history, return enriched object
  const habit = await this.firebase.get(`habits/${habitId}`);
  const fresh = (await this.firebase.getList(...)).filter((c) => c.userId === userId);
  return this.enrich(habit, fresh, todayKey);
}
```

`this.firebase` is the `FirebaseService` — its `.ref().set()` writes a JSON node into the Firebase Realtime Database. `getList('habitCheckins')` reads all check-ins (across all users) and we filter by `userId` in code.

Step 7 — Firebase persists the change

Firebase stores the JSON tree update on its servers. From now on, this data exists forever (until deleted) and is visible to any future request from this user.

Step 8 — Response travels back

The service returns the enriched habit object (habit + computed `streak`, `doneToday`, `scheduledToday`, `history[30]`). NestJS serializes it to JSON and sends it back over HTTP:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "abc-123",
  "title": "Wakeup",
  "doneToday": true,
```

```
  "streak": 1,
  ...
}
```

Step 9 — Frontend service receives the response

Back in `habit.service.ts`, the `.tap((u) => this.replaceHabit(u))` runs. The local `habits$` BehaviorSubject emits the new value. Every component subscribed to it (the dashboard and the habits page) re-renders with the updated streak / done state.

Step 10 — User sees the result

In about 200–500 ms (most of which was the network roundtrip), the UI shows:

- Checkbox filled
- Streak number bumped from 0 → 1
- Toast: "Nice — 1-day streak"

But thanks to the optimistic update in step 3, the checkbox actually flipped **instantly** (within ~16ms) when the click happened — the server response just confirmed it.

---

7. Authentication explained

Why JWT?

We need a way for the backend to know **which user** is sending a request. Options:

1. **Sessions** (server stores user-id ↔ session-id in memory/DB; browser sends a cookie). Simple but stateful — every server in a cluster needs access to the session store.
2. **JWTs** (the user's id is encoded into a signed token; backend verifies the signature). Stateless — any server can verify it with the secret key.

This app uses JWTs.

Two-token system

Token	Lifespan	Used for	Stored in
Access token	15 minutes	Every API call	`localStorage` (in browser)
Refresh token	7 days	Only for `/auth/refresh`	`localStorage` (in browser) + hashed in DB

Short-lived access tokens limit damage if one leaks. Refresh tokens are stored hashed on the backend so they can be revoked (logout invalidates them server-side).

Login flow

1. User submits email + password on `/auth/login`.
2. Frontend calls `POST /api/auth/login` with `{email, password}`.
3. Backend (`auth.service.ts`):
    - Looks up the user by email in Firebase
    - Compares the password with bcrypt against the stored hash
    - Generates two JWTs (access + refresh) signed with `JWT_SECRET` / `JWT_REFRESH_SECRET`
    - Stores the **hashed** refresh token in Firebase
    - Returns `{ user, accessToken, refreshToken }`
4. Frontend stores both tokens in localStorage and sets `currentUser` signal.
5. Router navigates to `/dashboard`.

Auto-attach token on every request

The HTTP interceptor (see step 4 above) automatically reads the access token from localStorage and adds it as a header on every outgoing request. You don't have to remember to do this in each service.

Auto-refresh when access token expires

The interceptor also catches 401 responses:

```
catchError((err: HttpErrorResponse) => {
  if (err.status === 401 && !isRefreshing && !req.url.includes('/auth/')) {
    isRefreshing = true;
    return auth.refresh().pipe(
      switchMap((res) => {                      // refresh succeeded
        isRefreshing = false;
        const retried = req.clone({ setHeaders: { Authorization: `Bearer ${res.accessToken}` } });
        return next(retried);                   // retry the original request
      }),
      catchError((refreshErr) => {              // refresh failed
        isRefreshing = false;
        auth.clearSession();                     // clear tokens, redirect to /auth/login
        return throwError(() => refreshErr);
      }),
    );
  }
  return throwError(() => err);
});
```

So if your access token expires while you're using the app, the user never notices — the next request gets a fresh access token transparently.

If even the refresh token is expired (you've been gone for a week), `auth.clearSession()` synchronously clears localStorage, resets the user signal, and navigates to `/auth/login`.

Route guards

`authGuard` checks `isLoggedIn()` (just "is there a token in localStorage?") before allowing access to `/dashboard`, `/tasks`, etc. If not, redirect to `/auth/login`.

`noAuthGuard` does the opposite for `/auth/*`: if you're already logged in, send you to `/dashboard`.

The `@Public()` decorator

The backend's `JwtAuthGuard` is registered globally — every endpoint requires a token by default. Endpoints like `/auth/register` and `/auth/login` mark themselves `@Public()` to opt out:

```
@Public()
@Post('login')
login(@Body() dto: LoginDto) { ... }
```

The guard checks for this decorator and skips the JWT check.

---

8. Feature deep-dives

8.1 My Plans (Tasks)

A "plan" is the generic term — it can be a task, trip, train trip, dinner reservation, meeting, event, or reminder. They all share one schema with a `type` field.

Backend (backend/src/tasks/):

- `TasksService.findAll(userId, filters)` — fetches tasks from Firebase, filters by userId, optionally by status/priority/type
- `create/update/remove/findOne` — standard CRUD
- `startTimer/stopTimer` — for the built-in time tracker on the detail page

Frontend:

- `TaskService` (`features/tasks/task.service.ts`) holds a `tasks$ BehaviorSubject`. Components subscribe to it.
- `TaskListComponent` ("My Plans" page) shows the list with:
  - Type filter chips (All / Task / Trip / Journey / Food / ...)
  - Status + priority dropdowns
  - **Pagination**: 15 per page, computed locally from `tasks$`

    - **DONE-last sorting**: completed tasks sink to the bottom, sorted by date desc among themselves
    - Edit/delete buttons + inline status dropdown
- `TaskFormComponent` (rendered inside `ModalComponent`) handles create/edit
- `TaskDetailComponent` (`/tasks/:id`) shows full info + a working timer

8.2 Goals

A **goal** has many **milestones**, each milestone has many **mini-goals**. Progress is auto-calculated from completed milestones.

Backend data shape:

```
goals/<id>      { title, userId, progress, ... }
milestones/<id> { goalId, status, order, ... }
minigoals/<id>  { milestoneId, goalId, status, ... }
```

This is a "flat" relational model on top of Firebase — milestones reference their parent goal by ID; we don't nest them in the JSON tree (so we can query milestones globally without loading every goal).

`GoalsService.findOne()` does the join in code: fetches the goal, fetches its milestones (filter by `goalId`), and for each milestone fetches its mini-goals (filter by `milestoneId`). For 1 user with <100 goals this is fast; at scale you'd want indexes.

Progress computation: every time a milestone is created/completed/deleted, the service recomputes `goal.progress = completed / total * 100` and writes it back.

Frontend:

- `GoalListComponent` — card per goal with progress bar
- `GoalDetailComponent` — accordion of milestones, each with mini-goal checkboxes
- Reorderable milestones via PATCH `/:goalId/milestones/reorder`
- Resources field on each goal (free-text array)

8.3 Daily Routine (Habits)

The most data-heavy module — let's go through it carefully because it has streaks, heatmaps, timezones, and backfilling.

Schema

```
habits/<id> {
  id, title, description,
  weekdays: [0..6],           // 0=Sun..6=Sat — which days the habit is scheduled
  startTime, endTime,         // "HH:MM" 24h — optional time window
  reminderEnabled: boolean,
  icon, color,
```

```
  archived: boolean,
  userId, createdAt, updatedAt
}

habitCheckins/<id> {
  id, habitId, userId,
  date: "YYYY-MM-DD",         // one check-in per (habit, date)
  completedAt: ISO timestamp
}
```

A check-in is an atomic record: "user X completed habit Y on date Z". Toggling on/off creates/removes one of these. The streak and heatmap are **computed at read time** by walking through check-ins — they're never stored as fields.

Streak computation

Walk backwards from "today":

- If the day is scheduled (its weekday is in `habit.weekdays`):
  - If a check-in exists for that date → `streak++`
  - If no check-in AND it's today → don't break (user hasn't checked in yet today)
  - If no check-in AND it's a past day → break (streak ends here)
- Stop scanning once cursor < habit.createdAt (or after 366 days as a safety cap)

This gives the **current** streak, which is the number of consecutive scheduled days completed up to today.

30-day heatmap

For each of the last 30 days, compute `{date, done, scheduled}`. The frontend renders these as small color-coded cells:

- Dim gray = not scheduled
- Faint red = scheduled but missed
- Green gradient = done (intensity from light to solid based on aggregate completion %)

Days **before** the habit was created are marked `scheduled: false` so they don't appear as "missed" (the user wasn't tracking yet).

Timezones — the tricky part

Firebase doesn't know about timezones. The backend originally used `new Date().toISOString().split('T')[0]` for "today" — but that's the **UTC** date. A user in India (UTC+5:30) at 12:01 AM Saturday would have the server still thinking it's Friday → habits scheduled for weekdays would appear scheduled "today" even on a Saturday morning.

Fix: the frontend sends `?today=YYYY-MM-DD` (computed from the user's local clock) on every habit API call. The backend uses that as the reference for streak/heatmap/scheduling. Check-ins are stored under whatever date the client says it is.

See `localTodayKey()` in `habit.service.ts` and `resolveToday()` in `habits.service.ts` (backend).

Backfilling past dates

The Daily Routine page has a date selector (← / → arrows) that lets you navigate back up to 29 days. When viewing a past date:

- The checklist shows habits scheduled for that day's weekday
- The check state comes from `habit.history[]` (the date's `done` cell)
- Clicking a checkbox calls `toggleDate(habitId, selectedDate)` instead of `toggleToday`
- Streak recomputes server-side; client doesn't try to predict it for past-day changes

Optimistic updates

The service flips local state synchronously before the HTTP call returns (see step 3 in the data flow walkthrough). Streak math is exact for "today" toggles (+1 or -1). For past-date toggles the streak isn't touched until the server responds (could ripple in non-obvious ways).

8.4 Calendar

A monthly grid view. The dashboard service has `/api/dashboard/calendar?year=2026&month=5` which returns every plan with a `dueDate` overlapping that month.

The frontend's `CalendarComponent` renders a 6×7 grid (Sun-Sat columns). Each day cell shows:

- Date number
- Office-hours overlay if it's a weekday (and not a holiday/leave)
- Colored chips for each plan, color-coded by `type` (PLAN_TYPES constant)
- Click a date → opens a modal with that day's full list + an "Add Plan" form

Holiday and leave toggles are stored locally per date.

Month picker

The month/year title in the calendar header is a clickable button. Clicking it opens a **month-picker dropdown** with:

- Previous/next year buttons (chevrons) to navigate years
- A 4×3 grid of month abbreviations (Jan–Dec)
- The currently-displayed month highlighted in accent color

Selecting a month jumps directly to it, reloads tasks and Google Calendar events for that month, and closes the picker. Clicking outside (the backdrop) also closes it. This avoids having to step through months one by one with the prev/next arrows.

8.5 Dashboard

The most aggregated page. It loads in parallel:

- `GET /api/dashboard/stats` — total tasks / completed today / active goals
- `TaskService.getToday()` — today's scheduled tasks
- `GoalService.loadAll()` — first 4 active goals for the progress section
- `HabitService.loadAll()` — all habits (for the Daily Routine widget)

The Daily Routine widget on the dashboard renders:

- **Progress ring** (SVG circle with `stroke-dasharray` animation) showing today's % completed
- **Three pill stats**: 30-day consistency %, best current streak, total habit count
- **Interactive checklist** — same UX as the habits page (click to toggle)
- **30-day aggregate heatmap** — for each day, the cell color shows what % of scheduled habits got done that day. Legend bar shows the gradient.

When you toggle a habit from the dashboard, the `HabitService.habits$` BehaviorSubject emits → both the dashboard widget and the habits page (if open in another tab) sync instantly because they both subscribe.

8.6 Journal

Day-end reflections. One entry per user per date, stored at `journal/<uuid>` in Firebase with fields `{userId, date (YYYY-MM-DD), title?, body, mood?, createdAt, updatedAt}`.

Why an upsert pattern? Since there's exactly one entry per (user, date), the API uses `PUT /api/journal/:date` instead of separate POST + PATCH. The backend finds an existing entry for that date and either updates or creates — same URL, idempotent. Saving twice is harmless.

Endpoints:

- `GET /api/journal` — list entries (date-desc, capped at 100)
- `GET /api/journal/:date` — fetch one date's entry (returns `null` if not yet written)
- `PUT /api/journal/:date` — upsert
- `DELETE /api/journal/:date` — remove

Frontend (features/journal/journal-page/journal-page.component.ts):

- A `selectedDate` signal, defaulting to today's local date
- Date navigation arrows (← / →) to walk back up to a year of past entries
- The form auto-fills from the cached `entries$` BehaviorSubject when `selectedDate` changes
- Mood emoji chips (8 options), title, body textarea
- Live word count, "✓ Saved" pill that fades after 2.5s
- Below the form: a list of past entries with date · mood · title · 2-line preview; clicking any switches `selectedDate` to view/edit that entry

Streak math (client-side computed signal): walk back from today's local date, counting consecutive days with entries. Capped at 365.

8.7 Projects (freelance tracker)

A freelance pipeline: each project moves through statuses (`LEAD → QUOTED → IN_PROGRESS → DELIVERED → PAID` , or branches to `LOST` / `ON_HOLD`). Payments are recorded against the project as separate records.

Schema:

```
projects/<id> {
  id, userId, title, clientName, clientContact, description,
  status, quotedAmount, currency, startDate, deadline, deliveredAt,
  progress, portfolioLinks: string[], archived, createdAt, updatedAt
}

projectPayments/<id> {
  id, projectId, userId, amount, currency, date, note?, method?, createdAt
}
```

Computed fields added by `enrich()` on every read (server-side, never stored):

- `payments[]` — all payments for this project, sorted date-desc
- `totalReceived` — sum of payments
- `balance` — `max(0, quotedAmount - totalReceived)`
- `isOverdue` — `deadline < today` AND status is not `PAID` or `LOST`

Endpoints under `/api/projects` :

- `GET /` — list (sorted "active work first" then by deadline)
- `GET /:id` , `POST /` , `PATCH /:id` , `DELETE /:id` — standard CRUD (delete cascades to payments)
- `POST /:id/payments` — record a payment
- `DELETE /:projectId/payments/:paymentId` — remove a payment

Status transitions that the service handles automatically:

- Moving to `DELIVERED` or `PAID` stamps `deliveredAt` if it's not set yet
- Moving back from `DELIVERED` clears `deliveredAt`

Frontend has three views:

1. **List page** (`/projects`) — stats summary (total, in-progress, outstanding balance, this-month income, pending quotes), status filter chips, project rows with status pill / deadline / amounts / progress bar. Active work auto-sorted to the top.
2. **Detail page** (`/projects/:id`) — header with title + status + overdue flag, a quick-controls card for status dropdown + progress slider, a 2-column layout below: left has description + portfolio links + dates, right has the payments section with an inline "+ Record payment" form and a list of past payments.
3. **Form** (modal) — title + client info + description + status + quoted amount + currency + dates + progress + portfolio links array.

Currency: defaults to INR but each project can override. Money is formatted with `en-IN` grouping (e.g. ₹1,25,000).

The "may or may not come" flow — leads that haven't converted yet sit in `LEAD` or `QUOTED` status. If they go cold, mark them `LOST` and they drop to the bottom of the list (with the LOST filter chip available to surface them later for follow-up).

8.8 Google Calendar integration (two-way)

The integration does two distinct things, in two directions:

Direction	Trigger	What happens
Push (app → Google)	User saves a task with a `dueDate`	Backend creates a Google event in the user's primary calendar; stores the returned event ID on the task
Pull (Google → app)	User opens `/calendar`	Backend lists events from ALL the user's subscribed Google calendars in the viewed month and returns them for read-only display

OAuth 2.0 flow

The user's Google account is connected via OAuth 2.0. The flow:

```
1. User clicks "Connect Google Calendar" in the sidebar
        │
        ▼  GET /api/google/auth
2. Backend generates a consent URL with scopes + redirect_uri + state=userId
        │
        ▼ returns { url }
3. Frontend redirects browser to Google's consent screen
        │
        ▼  user approves
4. Google redirects browser to our backend at GOOGLE_REDIRECT_URI
   with ?code=<auth_code>&state=<userId>
        │
        ▼  GET /api/google/callback
5. Backend exchanges the auth code for access + refresh tokens
   and stores them on the user record in Firebase
        │
        ▼  redirect to frontend dashboard
6. Frontend updates its connected signal to true
```

Why two tokens? Same reason as our app's auth — short-lived access tokens limit damage if leaked; long-lived refresh tokens stay safe in our DB and silently mint new access tokens.

Where tokens live: on each user record in Firebase as `googleAccessToken`, `googleRefreshToken`, `googleTokenExpiry`, `googleCalendarConnected`.

OAuth scopes we request

Scope	What it allows
`calendar.events`	Read + write events on calendars the user has access to (primary, etc.) — used by the push direction
`calendar.readonly`	Read events across all the user's subscribed calendars — used to pull festivals, holidays, third-party-app bookings
`calendar.calendarlist.readonly`	List the user's subscribed calendars (Holidays in India, Birthdays, work team calendars) so we can enumerate them before reading events

Scopes are bundled in the `getAuthUrl()` call inside `GoogleCalendarService`. If you change the list, **existing users must disconnect + reconnect** for their tokens to be valid for the new scope; Google won't quietly upgrade an old token's permissions.

Push direction (app → Google)

When a task with a `dueDate` is created:

1. `TasksService.create()` saves the task in Firebase
2. It then asynchronously calls `GoogleCalendarService.createEvent(userId, task)` (fire-and-forget — task creation never blocks on Google)
3. `createEvent` converts the task to Google's event format (`taskToEvent`): timed events use `start.dateTime` , all-day use `start.date` , color is mapped from `task.type`
4. The returned `googleEventId` is patched back onto the task in Firebase
5. On updates/deletes the same `googleEventId` is used to keep the Google event in sync

Each push uses a fresh authorized client via `getAuthorizedClient(userId)` , which:

- Reads the stored refresh token
- Constructs an `OAuth2Client` with the saved credentials
- Subscribes to the `'tokens'` event so any refreshed access tokens get persisted back to Firebase
- Hands the client to `google.calendar({...})`

Pull direction (Google → app)

When the calendar view opens, the frontend calls `GET /api/google/events?from=YYYY-MM-DD&to=YYYY-MM-DD` . The backend:

1. Builds a UTC time range (inclusive `from` , exclusive end-of- `to` +1 because Google's `timeMax` is exclusive)
2. Calls `calendarList.list()` to enumerate all the user's subscribed calendars. If the scope is insufficient, falls back to `[{ id: 'primary' }]`
3. Calls `events.list()` on each calendar in parallel (`Promise.all`), with `singleEvents: true` to expand recurring events into individual instances
4. Builds a `Set<string>` of `googleEventId` s currently saved on the user's tasks (so events we ourselves pushed are filtered out — no duplicates)
5. Flattens, normalizes each event:

- All-day events: `{start, end}` are `YYYY-MM-DD` (end converted to inclusive — Google's all-day end is exclusive)
- Timed events: `startTime` and `endTime` are local `HH:MM`

6. Sorts ascending by date then start time
7. Returns the array

The frontend's calendar component takes the response and, for each day cell, picks events whose `start <= dayKey && end >= dayKey` (so a multi-day festival shows on every day in its range).

Token refresh + invalid_grant handling

If an access token is expired but the refresh token is still valid, `googleapis` silently refreshes it and emits a `'tokens'` event. We listen to that and persist the new access token + expiry to Firebase.

If the **refresh token itself is dead** (revoked by the user, expired due to inactivity, or invalidated because we changed scopes), Google returns `invalid_grant`. `handleAuthError(userId, err)` detects this and **clears the user's Google connection in Firebase** — setting `googleCalendarConnected: false` — so the UI prompts a reconnect on the next status check, instead of silently failing forever.

Common setup issues

Error	Cause	Fix
`Error 400: redirect_uri_mismatch`	The `GOOGLE_REDIRECT_URI` env var doesn't exactly match what's listed in the Google Cloud OAuth client's "Authorized redirect URIs"	Add `http://localhost:3000/api/google/callback` to the client in Google Cloud Console
`invalid_grant` (in backend logs)	Refresh token revoked or scope-mismatched	User must disconnect + reconnect once
Toggle reads "G Cal on" but events don't appear	Same as above — token is stored but dead	Per-calendar API errors that are auth-related (401 / invalid_grant / invalid_token) now re-throw so the outer catch runs `handleAuthError` and clears the connection. User reconnects. Diagnostic logs print `[gcal] listEvents user=… calendars=N fetched=M` so you can see the call land
Festivals / birthdays not pulled in	Scope insufficient (only `calendar.events` was granted, not `calendar.readonly` + `calendar.calendarlist.readonly`)	Disconnect + reconnect; approve all permissions on the consent screen

> **Why 403 isn't treated as a connection-killer**: Google returns 403 for per-calendar permission denials too (e.g. you don't own one of the subscribed calendars). Auto-clearing on 403 would kill the whole connection because of one bad calendar. So `isAuthError` only matches 401 / invalid_grant / invalid_token.

8.9 Trips (Kanban)

A Kanban board for travel plans with four lanes: **Bucket List** → **Planning** → **Booked** → **Visited**. Cards are drag-and-droppable between columns. Clicking a card opens an edit modal with a Delete button (red, bottom-left).

Viewport-height layout

The board uses a full-viewport-height layout: the four lanes fill the available screen height (no page-level scroll), and cards scroll **inside each lane independently**. This is achieved by:

- `:host { height: 100% }` on the component propagates the computed height from `.layout-content`
- `.board-scroll { flex: 1; overflow: auto }` fills remaining height and scrolls the whole board horizontally when all four lanes can't fit
- Each `.lane { overflow: hidden }` and `.lane-cards { flex: 1; overflow-y: auto; min-height: 0 }` confine card scroll to the lane itself

The layout shell adds `overflow: hidden` to `.layout-content` when on a board route (`/trips` or `/buy-list`) to prevent the page from double-scrolling.

Card sort order

Cards within each lane are automatically sorted:

Lane	Sort
Bucket List	Alphabetical A-Z by title
Planning	Nearest upcoming date first, no-date entries last
Booked	Nearest upcoming date first, no-date entries last
Visited	Newest first (descending by start date), no-date entries last

Card display

Each dated card shows a **year pill** (e.g. `2026`) instead of a duration — the duration was misleading for open-ended or single-day trips.

Schema:

```
trips/<id> {
  id, userId, title, destination, description,
  status: BUCKET | PLANNING | BOOKED | VISITED | CANCELLED,
  startDate, endDate,
  companions, budget, currency,
  notes, references: string[],  // inspiration links (reels, blogs)
  taskId: string | null,        // FK to auto-created task in /tasks/*
```

```
  createdAt, updatedAt
}
```

Endpoints under `/api/trips`: standard CRUD (`GET`, `POST`, `PATCH /:id`, `DELETE /:id`).

Calendar sync (the "smart" part)

Any trip with a `startDate` AND status in `PLANNING | BOOKED | VISITED` gets a **linked task** in the `/tasks/*` collection (type='trip', dueDate=startDate, endDate=endDate, location=destination, description=notes). The trip's `taskId` field points to that task.

What this unlocks for free:

- The linked task shows on `/calendar` with the regular task chip
- The calendar cell shows a `✈ Trip` overlay (similar to office hours / leave tags)
- The habit module's trip-day auto-exclusion (see 8.3) sees the trip and pauses streaks for those dates
- The plan also appears in `/tasks` (My Plans), so the user can edit it like any other task

Sync behavior:

- **Create / move to Planning|Booked|Visited with dates** → `syncLinkedTask` creates or updates the task
- **Move back to Bucket List or Cancel** → the existing task is deleted, `taskId` cleared (no orphan plan on calendar)
- **Delete the trip** → linked task is also deleted (cascade)
- **First read after this feature shipped** → `findAll` runs a one-time backfill: any trip that should have a plan but doesn't (created under the old "only BOOKED" rule) gets its task created automatically

Auto-promote past trips to Visited

Every `findAll` scans for BOOKED trips whose end date (or start date if no end) is before today and writes them back as `VISITED`. So if the user forgets to drag a finished trip to the Visited column, it'll be there on the next page load.

Drag-and-drop (native HTML5, no library)

Each card sets `draggable="true"` with `dragstart` / `dragend` handlers; each lane has `dragover` / `drop` handlers. State held in two signals: `draggingId` (which card is being dragged) and `draggingOver` (which lane the cursor is in). When dropped on a different column, the trip's status is updated via the regular `PATCH /api/trips/:id` — same path used by the form. Visual feedback: dragging card becomes 40% transparent and rotates 2°; the target lane gets a green dashed outline.

Bug history: the "disappearing trip"

Initial release had a bug where trips saved with no inspiration links would disappear after a page reload. Root cause: Firebase Realtime DB drops empty arrays on write (`references: []` is stored as no-field). On read-back, `references` was `undefined`. The frontend template hit `t.references.length` and threw inside

`@for`, silently dropping the whole card from the render. Fixed by adding a `normalize()` helper in the service that backfills `references: trip.references ?? []` on every read/create/update response.

8.10 Buy List (Kanban)

A Kanban board for things you need to buy. Four lanes: **Want** → **Considering** → **Bought** → **Skipped**. Same UX as Trips (drag-and-drop, click-card-to-edit, delete button inside the form), but simpler — no calendar integration, no auto-promotion.

The board uses the same **viewport-height lane layout** as Trips — lanes fill available screen height, cards scroll internally within each lane, and no page-level scroll occurs on the board itself.

Schema:

```
buyItems/<id> {
  id, userId, name, category,
  status: WANT | CONSIDERING | BOUGHT | SKIPPED,
  urgency: LOW | MEDIUM | HIGH,
  estimatedPrice, boughtPrice, currency,
  store, link, notes,
  boughtAt: string | null,
  createdAt, updatedAt
}
```

Endpoints under `/api/buy-list`: standard CRUD.

Behavior nuances:

- Moving an item to **Bought** auto-stamps `boughtAt = today` if the client didn't specify one. Moving away from Bought clears `boughtAt` + `boughtPrice` (so they don't linger as stale data).
- Cards show estimated price for unbought items, actual paid price for bought items, an urgency pill (color-coded HIGH/MEDIUM/LOW) for active items, and a 🔗 quick-link if a product URL is set.
- Sort order: WANT → CONSIDERING → BOUGHT → SKIPPED; within each, HIGH urgency first, then newest.

---

9. Recurring patterns you'll see in the code

Standalone Angular components

Every component declares `standalone: true` and lists its own imports:

```
@Component({
  selector: 'app-habit-list',
  standalone: true,
```

```
  imports: [ModalComponent, HabitFormComponent, ...],
  template: `...`,
  styles: [`...`],
})
export class HabitListComponent { ... }
```

No NgModule needed — you just import the component class wherever you use it.

Signals + computed properties

Modern Angular reactive primitives:

```
habits = signal<Habit[]>([]);           // mutable state
todayHabits = computed(() =>            // derived state, auto-updates when habits() changes
  this.habits().filter(h => h.scheduledToday)
);

// Usage in template: {{ todayHabits().length }}
// Usage in code: this.habits.set([...new array]);
```

BehaviorSubject for cross-component state

```
@Injectable({ providedIn: 'root' })  // singleton
export class HabitService {
  habits$ = new BehaviorSubject<Habit[]>([]);

  loadAll() {
    return this.http.get<Habit[]>(...).pipe(
      tap((h) => this.habits$.next(h))   // emit new value to all subscribers
    );
  }
}

// In component:
this.habitService.habits$.subscribe((habits) => this.habits.set(habits));
```

Any component that subscribes to `habits$` will see updates when any other component (or the service itself) calls `.next(...)`. This is how the dashboard and the habits page stay in sync.

HttpClient + observables

All HTTP calls return RxJS `Observable<T>`:

```
this.http.get<Habit[]>('/api/habits').subscribe({
  next: (habits) => { /* success */ },
  error: (err) => { /* failure */ },
});
```

You can chain operators with `.pipe()`:

- `tap((x) => ...)` — side effect, doesn't change the value
- `map((x) => transform(x))` — transform the value
- `catchError((err) => ...)` — handle errors, return a fallback or rethrow

Reactive Forms

For any form:

```
form = this.fb.group({
  title: ['', Validators.required],
  description: [''],
});

// In template:
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="title" />
  <input formControlName="description" />
  <button type="submit" [disabled]="form.invalid">Save</button>
</form>
```

`form.value` is `{title, description}`; `form.valid` / `invalid` reflects validator state.

NestJS controllers + services + DTOs

The same shape repeats for every backend feature:

Controller — just routes, no logic:

```
@Controller('habits')
export class HabitsController {
  constructor(private habitsService: HabitsService) {}
  @Get() findAll(@CurrentUser('id') userId: string) {
    return this.habitsService.findAll(userId);
  }
}
```

Service — actual business logic + Firebase reads/writes:

```
@Injectable()
export class HabitsService {
  constructor(private firebase: FirebaseService) {}
  async findAll(userId: string) {
    const habits = await this.firebase.getList<Habit>('habits');
    return habits.filter((h) => h.userId === userId);
  }
}
```

DTO — incoming body validation:

```
export class CreateHabitDto {
  @IsString() @MaxLength(120) title: string;
  @IsOptional() @IsArray() weekdays?: number[];
  ...
}
```

The global `ValidationPipe` automatically validates `@Body() dto: CreateHabitDto` against these decorators and returns 400 with a useful error if it fails.

Optimistic updates

When a click should feel instant but needs a server roundtrip:

1. Flip the local state immediately
2. Fire the HTTP call
3. On response: replace local state with the server's authoritative version
4. On error: revert the local state

See `HabitService.toggleToday()` for the pattern.

---

10. Adding a new feature module

Suppose you want to add a "Notes" feature.

Backend

1. **Create the folder**: `backend/src/notes/`
2. **DTO** — `dto/create-note.dto.ts`:

```
import { IsString, IsOptional, MaxLength } from 'class-validator';
export class CreateNoteDto {
```

```
  @IsString() @MaxLength(200) title: string;
  @IsOptional() @IsString() body?: string;
}
```

3. **Service** — `notes.service.ts`:

```
@Injectable()
export class NotesService {
  constructor(private firebase: FirebaseService) {}
  async findAll(userId: string) {
    return (await this.firebase.getList<any>('notes')).filter(n => n.userId === userId);
  }
  async create(userId: string, dto: CreateNoteDto) {
    const id = randomUUID();
    const note = { id, ...dto, userId, createdAt: new Date().toISOString() };
    await this.firebase.ref(`notes/${id}`).set(note);
    return note;
  }
}
```

4. **Controller** — `notes.controller.ts`:

```
@Controller('notes')
export class NotesController {
  constructor(private service: NotesService) {}
  @Get() findAll(@CurrentUser('id') userId: string) { return this.service.findAll(userId); }
  @Post() create(@CurrentUser('id') userId: string, @Body() dto: CreateNoteDto) {
    return this.service.create(userId, dto);
  }
}
```

5. **Module** — `notes.module.ts`:

```
@Module({ controllers: [NotesController], providers: [NotesService] })
export class NotesModule {}
```

6. **Wire into** `app.module.ts`: add `NotesModule` to the `imports` array.

That's it — backend is done. Test with curl: `curl http://localhost:3000/api/notes -H "Authorization: Bearer ..."`.

Frontend

1. **Model** — `frontend/src/app/core/models/note.model.ts`:

```
export interface Note { id: string; title: string; body: string | null; userId: string; creat
```

2. **Service** — `frontend/src/app/features/notes/note.service.ts`:

```
@Injectable({ providedIn: 'root' })
export class NoteService {
  private http = inject(HttpClient);
  notes$ = new BehaviorSubject<Note[]>([]);
  loadAll() { return this.http.get<Note[]>(`${environment.apiUrl}/notes`).pipe(tap(n => this.
  create(dto: Partial<Note>) { return this.http.post<Note>(`${environment.apiUrl}/notes`, dto
}
```

3. **List component** — render the notes from `noteService.notes$ | async`.
4. **Add a route** in `app.routes.ts`:

```
{
  path: 'notes',
  loadComponent: () => import('./features/notes/note-list/note-list.component').then(m => m.N
}
```

5. **Add a sidebar link** in `sidebar.component.ts`.

Done. Roughly mirror the structure of `habits/` or `goals/` and you'll be consistent with the rest of the codebase.

---

11. Running the project locally

Prerequisites

- Node.js 18 or higher (`node --version`)
- A Firebase project with Realtime Database enabled
- A Firebase service account key file

Backend setup

1. `cd backend && npm install`
2. Put your Firebase service account JSON at `backend/serviceAccountKey.json` (download from Firebase Console → Project Settings → Service Accounts → "Generate new private key")
3. Create `backend/.env`:

```
JWT_SECRET="any-random-32-char-string"
JWT_REFRESH_SECRET="a-different-random-32-char-string"
FIREBASE_DATABASE_URL="https://YOUR-PROJECT-default-rtdb.firebaseio.com"
```

4. `npm run start:dev` — starts the backend with hot reload on `http://localhost:3000/api`

Frontend setup

1. `cd frontend && npm install`
2. `npm start` — runs `ng serve` and opens `http://localhost:4200`

Common issues

Symptom	Fix
`EADDRINUSE: address already in use :::3000`	A previous backend process didn't die. Find the PID: `netstat -ano | grep :3000` then kill it.
Login fails immediately	Backend isn't running, or `JWT_SECRET` changed since the user was created (tokens issued before become invalid).
Habits show on wrong day	Make sure the frontend is sending `?today=...` — see `HabitService.todayParam()`.
Frontend can't reach backend	CORS issue. Backend must list `http://localhost:4200` in `app.enableCors({origin: [...]})`.
Firebase connection error on backend start	`serviceAccountKey.json` missing or `FIREBASE_DATABASE_URL` wrong.

---

12. Concepts used in this project

A scannable catalog of every framework feature, language feature, and architectural pattern this project actually uses. Each entry has a one-line "what it is", a file pointer, and a short code example so you can find it in the codebase.

12.1 Frontend (Angular) concepts

Components & structure

Standalone components — Angular 19's preferred component style. The component declares its own imports list; no `NgModule` boilerplate.

- Used everywhere: every `.component.ts` in `frontend/src/app/`.
- Example:

```
@Component({
  selector: 'app-habit-list',
  standalone: true,
  imports: [ModalComponent, HabitFormComponent, ConfirmDialogComponent],
  template: `...`,
})
export class HabitListComponent { }
```

Inline templates and styles — `template:` / `styles:` arrays are inlined inside the `@Component()` decorator. (Larger components could use `templateUrl` / `styleUrls`, but this project keeps everything in one file.)

- Used in every component.

Component lifecycle hooks — methods Angular calls at specific points in a component's life.

- `ngOnInit()` — runs once after the component is created. Used to load data and subscribe to streams.
- `ngOnDestroy()` — runs when the component is removed. Used to unsubscribe.
- `ngOnChanges(changes)` — runs whenever an `@Input()` value changes.
- See `frontend/src/app/features/habits/habit-list/habit-list.component.ts`.

`@Input()` / `@Output()` / `EventEmitter` — how a parent component passes data into a child (`@Input`) and how a child notifies the parent (`@Output` + `EventEmitter`).

- Used in form components: `habit-form.component.ts`, `task-form.component.ts`.
- Example:

```
@Input() habit: Habit | null = null;
@Output() saved = new EventEmitter<void>();
// Parent: <app-habit-form [habit]="editing" (saved)="onSaved()" />
```

Content projection (`<ng-content>`) — lets a parent template inject children into a slot inside the child component's template. Used by the modal to wrap arbitrary form bodies.

- See `frontend/src/app/shared/components/modal/modal.component.ts`.
- Example:

```
<!-- modal.component.ts -->
<div class="modal-body"><ng-content /></div>

<!-- usage in parent -->
<app-modal [isOpen]="showForm">
```

```
  <app-habit-form [habit]="editing" />
</app-modal>
```

Templates & control flow

Property binding `[prop]="expr"` — sets an element/component property from a TS expression.

- Example: `<div [style.background]="h.color">`

Event binding `(event)="handler($event)"` — runs a TS method when a DOM event fires.

- Example: `<button (click)="toggleHabit(h)">`

String interpolation `{{ expression }}` — renders a TS expression as text.

- Example: `<span>{{ h.streak }}</span>`

New control flow `@if` **/** `@else` **/** `@for` — Angular 17+ built-in template syntax (no more `*ngIf` / `*ngFor`).

- `@for` requires a `track` expression for diffing.
- Example:

```
@if (todayHabits.length === 0) {
  <p>No habits today.</p>
} @else {
  @for (h of todayHabits; track h.id) {
    <div>{{ h.title }}</div>
  }
}
```

Pipes — `{{ value | pipeName }}` — transform a value in the template. Built-in pipes used: `DatePipe`, `DecimalPipe`, `AsyncPipe`.

- Example: `{{ today | date:'EEEE, MMMM d, y' }}`, `{{ progress | number:'1.0-0' }}`.

State & reactivity

Signals (`signal()`, `computed()`) — Angular 16+ reactive primitives. A signal holds a value and notifies dependents when it changes. `computed()` derives new signals from existing ones.

- Used in `dashboard.component.ts` for `habits`, `todayHabits`, `consistency30`, etc.
- Example:

```
habits = signal<Habit[]>([]);
doneToday = computed(() => this.habits().filter(h => h.doneToday).length);
// Read: this.doneToday()
// Write: this.habits.set([...newArray]);
```

RxJS `Observable` — a stream of values over time. HTTP calls return Observables. Subscribe with `.subscribe()` to react to each emission.

- Used in every service.
- Example: `this.http.get<Habit[]>(url).subscribe({ next: (h) => ... });`

`BehaviorSubject` — a special Observable that holds the *current* value and emits it to every new subscriber. Used as the in-memory cache for entity lists.

- Used in `TaskService`, `GoalService`, `HabitService`, `CategoryService`.
- Example:

```
habits$ = new BehaviorSubject<Habit[]>([]);
// emit a new value:  this.habits$.next([...new array])
// read latest sync: this.habits$.value
// subscribe:        this.habits$.subscribe(h => ...)
```

RxJS operators — chained with `.pipe()` to transform/side-effect on streams.

- `tap(x => ...)` — perform a side effect, don't change the value. Used to update BehaviorSubjects after HTTP calls.
- `catchError(err => ...)` — intercept errors, return a fallback Observable or rethrow.
- `switchMap(x => newObservable)` — replace the stream with a new one (used in interceptor to retry after token refresh).
- `map(x => transform(x))` — transform values.
- `throwError(() => err)` — create an erroring Observable.
- See `frontend/src/app/core/interceptors/auth.interceptor.ts`.

`AsyncPipe` — auto-subscribes to an Observable in the template and emits its values. Cleans up on destroy. (We mostly use signals now but a few places still use this.)

- Example: `<div>{{ tasks$ | async }}</div>`

Forms

Reactive Forms — form state lives in TS as a `FormGroup` / `FormControl` object with validators. The template binds via directives.

- Used in every form: `habit-form`, `task-form`, `goal-form`, `login`, `register`, `profile`.
- Building blocks: `FormBuilder`, `FormGroup`, `FormControl`, `Validators`, `ReactiveFormsModule`.

- Example:

```
form = this.fb.group({
  title: ['', Validators.required],
  description: [''],
  weekdays: [[]],
});
// template:
// <form [formGroup]="form" (ngSubmit)="submit()">
//   <input formControlName="title" />
// </form>
```

FormsModule (`ngModel`) — older template-driven forms with two-way binding `[(ngModel)]="prop"`. Used sparingly (e.g., calendar's holiday toggles).

Routing

Routes config — `frontend/src/app/app.routes.ts` defines all URL → component mappings.

Lazy loading — `loadComponent: () => import('...').then(m => m.X)` defers loading the component's JS chunk until the user navigates there.

- Cuts the initial bundle size dramatically.
- Used for every page route.

`RouterLink` / `RouterLinkActive` — directives for navigating without a page reload and highlighting the active link.

- Used in `sidebar.component.ts`.
- Example: `<a routerLink="/habits" routerLinkActive="active">Daily Routine</a>`

Route guards — functions that return `true` / `false` to allow or block navigation.

- `authGuard` blocks unauth users from `/dashboard`, `/tasks`, etc.
- `noAuthGuard` blocks logged-in users from `/auth/login`.
- See `frontend/src/app/core/guards/`.

`Router` + `ActivatedRoute` services — programmatic navigation and reading query params.

- Example (dashboard handles Google OAuth callback):

```
if (this.route.snapshot.queryParams['gcal'] === 'connected') {
  this.toast.success('Google Calendar connected!');
  this.router.navigate([], { queryParams: {}, replaceUrl: true });
}
```

HTTP

`HttpClient` — Angular's built-in HTTP client. Returns Observables, supports generics for typed responses.

- Example: `this.http.get<Habit[]>('/api/habits')`

Functional HTTP interceptors — pure functions that wrap every outgoing request. The `provideHttpClient(withInterceptors([authInterceptor]))` registration in `app.config.ts` plugs them in.

- See `auth.interceptor.ts`. It attaches the `Authorization` header and handles 401 refresh.

Query parameters & URL building — appended via template strings.

- Example: `${this.base}/habits?today=${this.localTodayKey()}`

Dependency injection

`@Injectable({ providedIn: 'root' })` — marks a class as a service the DI system can create. `'root'` makes it a singleton shared by the whole app.

- Used by every service: `AuthService`, `HabitService`, `ThemeService`, etc.

`inject(Service)` **function** — modern alternative to constructor injection. Works inside the class body OR inside functional interceptors/guards.

- Example: `private http = inject(HttpClient);`

Constructor injection — older style still used in some classes.

- Example: `constructor(private http: HttpClient, private router: Router) {}`

Singleton scope — `providedIn: 'root'` services are created once and shared. That's why a `BehaviorSubject` inside a service stays in sync across components.

App configuration

`ApplicationConfig` — defines what providers the app needs at bootstrap. Lives in `app.config.ts`.

- Example:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
```

```
  ],
};
```

`bootstrapApplication` — the standalone-app entry point, called in `main.ts`. Replaces the old `platformBrowserDynamic().bootstrapModule(AppModule)`.

Styling

CSS custom properties (variables) — theme tokens like `--accent`, `--bg-card`, `--text-primary` defined in `frontend/src/styles.css`. Components reference them via `var(--accent)`.

- Used in every component's `styles:` block.

Component-scoped styles — Angular's default view encapsulation rewrites selectors so component styles don't leak. You write `.btn-primary` and it becomes `.btn-primary[_ngcontent-xyz]` at runtime.

Dark/light theme via `data-theme` attribute — `ThemeService` sets `data-theme="dark"` on `<body>`; CSS variables override based on this attribute.

- See `frontend/src/app/core/services/theme.service.ts`.

Browser APIs used

`localStorage` — persistent key-value storage in the browser.

- `TokenStorageService` stores JWTs.
- `ThemeService` stores theme preference.

`Notification` **API** — declared (the habit form has a reminder toggle), but actual delivery is not yet implemented.

`environments/` — Angular's mechanism for swapping config per build (dev vs prod API URL).

12.2 Backend (NestJS) concepts

Module system

`@Module()` **decorator** — wraps a feature's controllers, providers (services), and imports into a single unit.

- Used in every feature: `habits.module.ts`, `goals.module.ts`, etc.
- Example:

```
@Module({
  controllers: [HabitsController],
  providers: [HabitsService],
})
export class HabitsModule {}
```

Root `AppModule` — imports every feature module and registers global providers like the JWT guard.

- See `backend/src/app.module.ts`.

Controllers & routing

`@Controller('prefix')` — declares a class as a HTTP controller and prepends a path prefix.

- Combined with `app.setGlobalPrefix('api')` in `main.ts`, `@Controller('habits')` serves `/api/habits/*`.

Method decorators: `@Get()`, `@Post()`, `@Patch()`, `@Delete()` — map a method to an HTTP verb. Path patterns can include `:param` segments.

- Example: `@Patch(':id') update(@Param('id') id: string, @Body() dto: UpdateHabitDto) { ... }`

Parameter decorators:

- `@Body() dto` — extracts the JSON request body
- `@Param('name')` — extracts a URL segment
- `@Query('name')` — extracts a query string value
- Used in every controller.

Implicit status codes — `200` for `GET/PATCH`, `201` for `POST`. Throw an exception to return another status.

Services & dependency injection

`@Injectable()` — same idea as Angular: marks a class as injectable. NestJS resolves dependencies from the module's `providers` array.

- Used in every service.

Constructor injection — declare what you need in the constructor.

- Example: `constructor(private firebase: FirebaseService) {}`

Singleton by default — every provider is a singleton inside its module unless you opt out.

Validation

DTOs (Data Transfer Objects) — plain classes describing what a request body should look like.

- Example: `backend/src/habits/dto/create-habit.dto.ts`

`class-validator` decorators — declarative validation rules.

- `@IsString()`, `@IsOptional()`, `@IsBoolean()`, `@IsInt()`, `@IsArray()`, `@IsDateString()`
- `@Min(n)`, `@Max(n)`, `@MaxLength(n)`, `@ArrayMinSize(n)`, `@ArrayMaxSize(n)`
- `@Matches(regex)` — for custom string patterns (e.g. `HH:MM` time)
- Example:

```
export class CreateHabitDto {
  @IsString() @MaxLength(120) title: string;
  @IsOptional() @Matches(/^([01]\d|2[0-3]):[0-5]\d$/) startTime?: string;
}
```

Global `ValidationPipe` — installed in `main.ts` with `whitelist: true, forbidNonWhitelisted: true, transform: true`. Automatically rejects requests with unknown fields and converts incoming JSON into typed DTO instances.

`PartialType` (from `@nestjs/mapped-types`) — generates a DTO where all fields are optional. Used for update DTOs.

- Example: `export class UpdateHabitDto extends PartialType(CreateHabitDto) {}`

Authentication

Passport strategies — pluggable auth strategies. This project has two:

- `JwtStrategy` — validates the access token in the `Authorization` header.
- `JwtRefreshStrategy` — validates the refresh token in the request body (only used by `/auth/refresh`).
- See `backend/src/auth/strategies/`.

JWT signing — `JwtService.signAsync(payload, { secret, expiresIn })` creates a signed token.

`bcrypt` — one-way password hashing. We never store the plain password — only the bcrypt hash.

- `bcrypt.hash(password, salt)` on register.
- `bcrypt.compare(input, storedHash)` on login.

Refresh token rotation — every successful refresh issues a NEW refresh token (and stores its hash). Old refresh tokens become invalid. Limits replay attacks.

Authorization & permissions

Global guards — registered via the `APP_GUARD` provider so they run on every request.

- `JwtAuthGuard` is registered globally in `app.module.ts`; every endpoint requires a valid JWT unless marked `@Public()`.

Custom guards extending Passport's `AuthGuard`:

- `JwtAuthGuard` (`backend/src/auth/guards/jwt-auth.guard.ts`) — the global one. Skips routes with `@Public()` metadata.
- `JwtRefreshGuard` — only used on `/auth/refresh`.

`@Public()` decorator — sets metadata that `JwtAuthGuard` reads via `Reflector.getAllAndOverride()` to opt-out.

- See `backend/src/common/decorators/public.decorator.ts`.

Ownership checks — pure code-level "does this user own this record?" verification inside each service.

- Pattern: `private async ensureOwnership(userId, recordId)` throws `ForbiddenException` if userId doesn't match the record's `userId` field.
- Used in `HabitsService`, `GoalsService`, `TasksService`, etc.

Authentication vs Authorization — two different concepts:

- *Authentication* = "who are you?" → handled by `JwtAuthGuard` validating the token.
- *Authorization* = "are you allowed to do this?" → handled inside services via ownership checks.

Custom decorators

`@CurrentUser('id')` — extracts the authenticated user (or one of its fields) from the request, set there by the JWT strategy.

- See `backend/src/common/decorators/current-user.decorator.ts`.
- Used in every controller: `@CurrentUser('id') userId: string`.

`@Public()` — marks a route as bypassing the global guard. (See above.)

Exception handling

Built-in HTTP exceptions — throw them from services and Nest's exception filter turns them into proper HTTP responses.

- `NotFoundException` → 404
- `ForbiddenException` → 403
- `UnauthorizedException` → 401

- `BadRequestException` → 400
- `ConflictException` → 409 (e.g., email already taken)
- Example: `if (!habit) throw new NotFoundException('Habit not found');`

Configuration

`ConfigModule.forRoot({ isGlobal: true })` — loads `.env` once at startup; any service can read with `process.env.VAR` or `ConfigService`.

- Configured in `app.module.ts`.

`.env` file — holds secrets (`JWT_SECRET`, `FIREBASE_DATABASE_URL`) — never committed to git.

Middleware concepts

Pipes — run after routing, before the controller method. `ValidationPipe` is the only one we use globally.

Guards — run before controller methods. Returning false (or throwing) blocks the request.

Interceptors / Exception filters — Nest has these too; we don't use custom ones beyond the built-in exception filter.

CORS — `app.enableCors({ origin: [...], credentials: true })` in `main.ts` whitelists which frontend origins can call this API. Browsers block calls from origins not in the list.

Global prefix — `app.setGlobalPrefix('api')` means every controller path is automatically prefixed with `/api`.

Lifecycle

`OnModuleInit` — a hook that runs once after the module's dependencies are resolved. Used by `FirebaseService` to initialize the Firebase Admin SDK.

- See `backend/src/prisma/firebase.service.ts`.

`async` / `await` — every service method that touches Firebase returns a `Promise`. Standard ES2017 syntax.

`Promise.all([...])` — for parallel async calls. Used in `HabitsService.findAll` to fetch habits and check-ins simultaneously.

Crypto & ID generation

`crypto.randomUUID()` — Node's built-in to generate a random unique ID (used as Firebase keys for new records).

`bcrypt.hash` / `bcrypt.compare` — password hashing (see Authentication above).

12.3 Database (Firebase) concepts

NoSQL JSON tree

The database is just one giant JSON object. There are no tables, no rows, no columns, no SQL — just nested keys and values. Top-level keys act like "collections":

```
{
  "habits": {
    "abc-123": { "id": "abc-123", "title": "Wakeup", ... },
    "def-456": { ... }
  },
  "habitCheckins": { ... },
  "users": { ... }
}
```

You read/write at any path: `habits/abc-123/title` gives you just the title.

Firebase Admin SDK (server-side)

The backend uses `firebase-admin` — a privileged SDK that bypasses client-side security rules. It needs a service account key file (or env var) to authenticate.

- Initialized in `backend/src/prisma/firebase.service.ts`.

Service account credentials — `serviceAccountKey.json` is the backend's god-mode credential. Never expose it to clients or commit it to git.

Operations (via `FirebaseService`)

This project wraps the Admin SDK in a small service exposing 6 methods:

Method	What it does
`ref(path)`	Get a database reference to a path (low-level)
`get<T>(path)`	Read a single record, returns `null` if missing
`getList<T>(path)`	Read all children at a path as an array
`push(path, data)`	Add a new child with an auto-generated key
`update(path, data)`	Merge fields into an existing record (preserves other fields)

Method	What it does
`remove(path)`	Delete an entire path

Most services use `ref(path).set(record)` (overwrite) plus `update()` for partial changes.

No native joins or relations

Firebase can't join `goals` with `milestones`. You fetch both lists and combine them in JS code.

Example from `GoalsService.findOne`:

```
const goal = await this.firebase.get(`goals/${id}`);
const allMilestones = await this.firebase.getList('milestones');
const goalMilestones = allMilestones.filter(m => m.goalId === id);
```

No indexes (by default)

Real Firebase supports `.indexOn` security rules for filtering, but this project doesn't configure any. We fetch entire collections and filter in code. Fine for small data, slow at scale.

No automatic timestamps or schemas

Firebase doesn't validate field types or add timestamps. We manually:

- Generate `createdAt` and `updatedAt` strings: `new Date().toISOString()`
- Validate at the controller layer via DTOs

Denormalization

Children reference parents by ID, not by nesting. Example:

- A milestone has `{ id, goalId, ... }` — `goalId` is the foreign key.
- We don't store milestones *inside* the goal record.

Why: makes per-record updates atomic, makes queries simpler, avoids load-the-whole-thing-just-to-update-one-field.

Eventual consistency / derived data

Derived values (`streak`, `progress`, `consistency`) are computed at READ time, not stored. So they're never stale — but every read is more work.

Authentication is NOT done by Firebase

We don't use Firebase Auth or Firestore security rules. The backend authenticates users with its own JWT system and uses Firebase as a dumb data store. Frontend never touches Firebase directly.

Date storage as strings

All timestamps are stored as ISO 8601 strings (`"2026-05-23T10:30:00.000Z"`) — they sort lexicographically the same as chronologically and are human-readable.

Habit check-in dates use `YYYY-MM-DD` (no time component) so we can do equality checks for "did the user check in on this day?"

12.4 Cross-cutting / fullstack concepts

HTTP request/response cycle — browser sends a request (verb + URL + headers + body), server processes it, sends back a response (status code + headers + body). Stateless: each request is independent.

JSON — the data format both layers speak. Just text that looks like a JS object.

CORS (Cross-Origin Resource Sharing) — browser security rule. The frontend at `localhost:4200` is a different "origin" from the backend at `localhost:3000` . By default, the browser blocks cross-origin requests; the backend must explicitly opt in with `enableCors()` .

Stateless authentication — no server-side sessions. The server doesn't remember who you are between requests; it re-verifies the JWT on every call. Simplifies scaling (any server can handle any request).

Tokens vs sessions — Sessions store user identity server-side and use a cookie pointer. Tokens (JWTs) encode the identity itself, signed with a secret. We use tokens.

Refresh-token rotation — every refresh issues a new refresh token, invalidating the old one. Limits damage if a token leaks.

Optimistic UI updates — flip the UI immediately on user action, before the server confirms. Revert on error. Makes interactions feel instant despite network latency. Used in `HabitService.toggleToday` .

Environment variables — config values (API keys, DB URLs, secrets) injected at runtime via the OS environment. Different per machine; never committed.

TypeScript interfaces — shared shape definitions used by both client (`core/models/`) and parallel DTOs on the server (`dto/`). Compile-time safety.

Lazy loading — defer downloading code until it's needed. Reduces initial page load size. Used for every route in this app.

Hot module replacement (HMR) / watch mode — both Angular and NestJS dev servers watch for file changes and rebuild/restart automatically. You save a file, see results instantly.

Single Page Application (SPA) — the frontend is one HTML file that swaps content via JavaScript routing. No full page reloads between routes.

Dependency injection — the framework creates and supplies your dependencies instead of you newing them up manually. Makes testing and swapping implementations easier. Used heavily in both Angular and NestJS.

Reactive programming — model data as streams of values over time (RxJS) or as reactive cells that auto-recompute (Signals). The UI follows data automatically — no manual DOM updates.

Separation of concerns — controllers route, services hold logic, DTOs validate, components render, models type. Each piece does one thing.

---

13. Glossary

- **API**: A list of URLs (endpoints) the backend exposes for the frontend to call.
- **BehaviorSubject**: An RxJS class that holds a current value and re-emits it to new subscribers. Used as a local cache for entity lists.
- **CORS**: Cross-Origin Resource Sharing. A browser security rule that blocks frontend code on one domain from calling APIs on another unless the API allows it. The backend's `enableCors()` whitelists the frontend's origin.
- **DTO** (Data Transfer Object): A TypeScript class describing what shape a request body should have. Used by NestJS for validation.
- **Decorator**: A `@Something()` annotation in front of a class or method. Both Angular and NestJS use them heavily (`@Component` , `@Injectable` , `@Controller` , `@Post` , etc.).
- **Dependency injection**: A pattern where a class declares what it needs in its constructor (or via `inject()`) and the framework provides instances. Lets you write `private auth = inject(AuthService)` instead of `new AuthService()` .
- **Endpoint**: A specific URL + HTTP verb the backend handles, e.g. `GET /api/habits/today` .
- **Guard** (NestJS): A class that runs before a controller method to allow or block the request (e.g., check JWT).
- **Guard** (Angular Router): A function that runs before navigating to a route to allow or block it (e.g., redirect to login if not authenticated).
- **HTTP interceptor**: A function that wraps every outgoing HTTP request, often to add headers or catch errors globally.
- **JSON**: Plain-text format for structured data: `{"key": "value", "n": 42}` .
- **JWT** (JSON Web Token): A signed text token that encodes user identity. Sent as `Authorization: Bearer <token>` on every request.
- **Middleware**: Code that runs between request arrival and the controller handler (e.g., body parsing, CORS, auth).
- **Module** (NestJS): A class that bundles related controllers + services + imports.
- **Observable** (RxJS): A stream of values over time. `.subscribe(callback)` to react to each emission. HTTP calls return Observables.
- **Optimistic update**: Update the UI immediately on user action, before the server confirms. Revert on error.
- **Reactive Forms**: Angular's class-based form model with validators (`FormGroup` , `FormControl`).
- **REST API**: A style of HTTP API where URLs represent resources and verbs (GET/POST/PATCH/DELETE) represent operations. This project follows REST loosely.
- **Service** (Angular): An `@Injectable()` class for sharing logic/state across components.
- **Service** (NestJS): An `@Injectable()` class for business logic, separated from controllers.

- **Signal** (Angular): A reactive primitive `signal(initial)` with `.set()` / `.update()` and reactive `.computed()`.
- **Standalone component**: An Angular component that declares its own imports and doesn't need an NgModule.
- **State management**: How an app keeps its UI in sync with data. This project uses signals for component-local state and BehaviorSubjects in services for shared state.
- **`tap()`**: An RxJS operator that lets you do a side effect (like updating a BehaviorSubject) without changing the stream's value.

If something in the code isn't clear, search the file for the relevant decorator or service name — most things in this codebase follow a couple of repeating patterns, so once you understand one feature module, the others will read fast.